

邱汇迪

背景：四川大学视觉合成图形图像技术国家级重点实验室
邮箱：745008538@qq.com
电话：13540916673
个人网站：<https://ssgaryss.github.io>
Github：<https://github.com/ssgaryss>
简历更新时间：2026.8.4



过往经历

网易雷火 DJ02 项目组，游戏引擎开发，正式	2026.3 – 2026.9
腾讯光子工作室群，游戏客户端开发，实习	2024.10 – 2025.9
四川大学，计算机图形学，硕士	2023.9 – 2026.7
• CAD/CG 2026 中稿一篇(基于二维高斯泼溅的实时重光照算法) 校一等奖学金	
四川大学，计算机科学与技术(拔尖计划)，本科	2019.9 – 2023.7
• 专业前 5% 数学建模省级奖项 校级大创项目	

网易雷火 DJ02 期间

GI 探针烘焙流程重构和优化 [详细介绍](#) 2026.7

- 描述：原来方案烘焙管线继承自逆水寒手游，采用 UE4 Lightmass 作为烘焙器，完成后一键导入光图和 vlm 的方式实现 GI，并引入类似双主光等 Trick 方案解决远处没有 HLOD 时 LOD2 Culling 后光图的黑影问题。这个方案不适合本项目，因为本项目需要人物与环境融合(Lightmap 和 Lightprobe 对齐)，为快速解决当前场景 GI 的所有痛点(爆亮、偏色)，本人采用 AmbientCube 替代原本 UE SH 探针，将 Texture3D 贴图数量从 9 张 Texture3D 数据减少为 5 张 Texture3D (三张 X Y Z 方向数据和 DX12 光追烘焙的 Sky Visibility，一张 Dir + 亮度缩放，一张 Indirection)，并完全按照 UE VLM 稀疏探针的方式替代原本均匀探针，这样探针密度符合美术在局部的表现力。偏色问题是由于亮度范围不足导致，UE 是 HDR，本方案 RGBM 范围较小(实现了一个自动分析功能选择合适的亮度范围，并将超过的按照颜色比例 clamp 回范围，并正确指导美术打光缓解该问题)，过亮问题是美术打光问题(通过和美术积累的信任以及多次沟通，建立起了一个简单的打光原则)，其次 Shadowmask 采用一张主光 Lightdepth 解决，最终基本解决了效果问题。本来是要继续优化性能，设计探针流式加载功能，不采用全量加载 VLM 实现，做了一半项目没了.....

Triblend 地形材质系统和配套工具集 [详细介绍](#) 2026.4

- 摘要：原方案采用永劫手游 MatID 材质系统，为彻底解决其诸多编辑痛点，以及减少采样次数优化性能的目的。参考《Delta Force》GDC 方案设计了 Triblend 的地形材质系统并做出改进(一张 R16 贴图记录两个 ID 和 BotID-TopID 的权重，将浮点精度翻倍)，采样数量由 10 减少到不多于 9 次，不再需要美术选择材质通道，不再划分 16mx16m 网格区域，地形切分后保证 100% 材质连续。此外，支持材质旋转(如带方向的地砖，从而客户端可直接访问地形从而支持交互系统运作)，支持配套地形切分工具和检测工具(检测 mesh 下方是否有地形，如果存在需要扣掉提高性能)，支持一键从 MatID 迁移到 Triblend 地形，支持 Triblend 笔刷。改进方案在项目中经受住了时间的考验，全部场景均落地。

TerraKit 地编系统 [详细介绍](#) 2026.5

- 描述：地编系统完整版是草海编辑、地形编辑、PCG，其实只完成了草海编辑(地形编辑完成了一半，PCG 原计划是接实习生搬运的 UE PCG)，由于本项目美术需要逐 GO 微调每颗草，Unity 原生密度图不适用，项目骚操作采用 Unity 地形树笔刷画草，直到草的数量爆炸到十万，场景 YAML 文件超过 1GB 了，我采用 GrassBundle(草海集合)的设计，将每一批草单独序列化，然后编辑器下 GPU Instancing 渲染，支持美术点选/框选(将 GrassBundle 的草生成为 GO 任意编辑，并支持保存回去)，支持刷草(多种功能，还支持混刷和白名单等)，支持刷草色，支持擦除/去密/去重草，支持草烘焙(Raycast 取下方物体 Lightmap)，此外 GrassBundle 设计完美契合地编多人协作，所有 GrassBundle 可以按照任何规则划分/合并/重整等，同步 SVN 给所有人。最终是完美解决了这个问题，从 5 月后再无草海问题出现，得到了美术一致好评。

腾讯实习期间

最新游戏渲染逆向分析 [《使命召唤: 黑色行动 6》](#) [《无限暖暖》](#) 2025.6 and 2025.2

- 摘要：SMAA、TAA、NIS 超分、NLG(SIGGRAPH2024)、SSR、GTAO、PPLL OIT...

RDC-Parser [详细介绍](#) 2025.4

- 描述：RDC-Parser 是一个命令行工具，旨在解析 RenderDoc 的 rdc 文件为 JSON 与对应 Meshes, Textures 和 Shaders。

PUBGConfigUpgrader [详细介绍](#) 2025.3

- 描述：PUBGConfigUpgrader UE commandlet 工具，旨在将和平精英项目从公版 UE4 和 Galaxy 版 UE4 迁移到 UE5.4 时自动转换配置文件，且包含在原来版本上兼容更多机型等。

补充 详细了解我请访问我的[个人网站](#)，我非常擅长 RenderDoc 逆向，我有支持 UE 工具开发的经验，我对游戏有深厚的热情，我对图形学有深入的探究，我非常熟悉 OpenGL 以及有 Vulkan 使用经验，我对当前游戏渲染技术有宏观的了解。

个人项目

Pika Engine [网页链接](#) [代码链接](#)

2024.5 - 2024.10

- 描述：本人独立开发的渲染引擎，最能体现本人 C++ 编程能力，图形学知识，引擎理解和软件开发能力的开源项目。
- 摘要：
 - 1) 渲染：预定义材质、光源和阴影、Skybox。
 - 2) ECS：可以随意添加 Components，用 Entt 库进行 Entity 管理。
 - 3) 物理：2D 中已实现物理效果，用 Box2D 实现组件碰撞检测以及物理效果。
 - 4) 序列化：Scene 可以序列化到 YAML 文件并可以随时拖拽到 Viewport 进行渲染，支持 3D 文件导入导出。

专业技能

计算机语言方面：C++ 毫无疑问是我的主要语言，非常熟悉 STL，有独立开发完整开源项目的经验且对程序的合理架构有着较高的追求 ([Pika Engine](#))，临时抱佛脚地刷过上百道 LeetCode ([Leetcode_daily](#))，在编写项目的同时系统性看完《C++20 高级编程》，熟悉 C++20 新特性，擅长使用 Profile 工具（如 VS 自带的就很好用）对运行时间或内存进行监视，防止内存泄漏、低效代码等。

Python 也是我非常熟悉的语言，除了科研中大量使用，还开发过大量工具 ([RDC-Parser](#), [ModelGray](#))，了解 python 的局限性，有用 pybind 调用 C++ 底层实现多线程等大幅优化代码的经验。

图形学方面：研一学习并完成 GAMES101&202 课程与作业 ([GAMES-Renderer](#)是开发 Pika 前基于 GAMES 实现的一个引擎)。熟悉 Vulkan，完成 Vulkan Tutorial ([LetsVulkan](#))，热爱并擅长图形学（研一计算机图形学课程成绩 (97/100)），并具有较好数学能力（高数专业前 5%）。非常熟悉 OpenGL，看完 LearnOpenGL 并复现大部分内容。当前正在阅读《Real-Time Rendering, Fourth Edition》中。

RenderDoc：从开发 Pika 引擎开始我就开始用它逆向调试 shader 以及 profile 性能，实习期间又逆向过《使命召唤：黑色行动 6》和《无限暖暖》（可以点击链接查看我的分析文档）

Unreal Engine：除了[PUBGConfigUpgrader](#)外我还为项目组的减面插件实现了测试 PSNR、SSIM 评分并可视化的模块，有支持 UE 工具链的经验但谈不上专精，正在学习中。

英语方面：CET-6，有直接浏览英文的能力，可以非即兴英文汇报（项目组有纯英文开会，但我需要提前想一下怎么说不然可能会磕磕绊绊）